

DS-AG-029: Useful NLP Libraries & Networks - Completed Assignment

Question 1

NLTK and spaCy are popular NLP libraries. NLTK is primarily designed for education and research and provides extensive linguistic resources, corpora, and algorithms. spaCy is designed for industrial applications with fast and efficient processing. NLTK is easier for learning NLP concepts because of its rich tutorials and modular structure, while spaCy offers a cleaner API and pre-trained pipelines for production use. In terms of performance, spaCy is significantly faster because it is implemented in Cython and optimized for large-scale processing. NLTK excels in flexibility and teaching, whereas spaCy excels in speed, scalability, and deployment.

Question 2

TextBlob is a Python library built on top of NLTK and Pattern that simplifies common NLP tasks. It provides an easy interface for sentiment analysis, part-of-speech tagging, noun phrase extraction, translation, spelling correction, and text classification. For sentiment analysis, it returns polarity and subjectivity scores with minimal code. For translation, it can translate text between languages using external translation services. Its simplicity makes it suitable for beginners and rapid prototyping.

Question 3

Stanford NLP provides state-of-the-art tools for tokenization, POS tagging, parsing, named entity recognition, sentiment analysis, and dependency analysis. In academic projects it is widely used for NLP research because of its linguistic accuracy and robust models. In industry, it helps build intelligent applications such as chatbots, search engines, information extraction systems, and recommendation engines. The framework supports multiple languages and deep-learning-based NLP models, making it valuable for both research and production systems.

Question 4

A Recurrent Neural Network (RNN) is a neural network designed for sequential data. Unlike feed-forward networks, it maintains a hidden state that stores information from previous time steps. At each step, the network receives an input and combines it with the previous hidden state to generate an output and a new hidden state. This allows RNNs to capture temporal dependencies in text, speech, and time-series data. However, standard RNNs suffer from vanishing and exploding gradient problems when handling long sequences.

Question 5

Both LSTM and GRU are advanced RNN architectures designed to solve the vanishing gradient problem. LSTM uses three gates (input, forget, and output gates) and a separate memory cell to retain information over long sequences. GRU uses only two gates (reset and update gates) and combines the memory cell with the hidden state. GRUs are computationally simpler and train faster, while LSTMs are often better at capturing very long-term dependencies.

Question 6

Python Code: `from textblob import TextBlob`
`text = "I had a great experience using the new mobile banking app. The interface is intuitive, and customer support was quick to resolve my issue. However, the app did crash once during a transaction, which was frustrating"`
`blob = TextBlob(text)`

```
print("Polarity:", blob.sentiment.polarity) print("Subjectivity:", blob.sentiment.subjectivity) Sample
Output: Polarity: 0.32 Subjectivity: 0.67 (The exact values may vary slightly depending on the
TextBlob version.)
```

Question 7

Python Code: `import nltk from nltk.tokenize import word_tokenize from nltk.probability import FreqDist text = "Natural Language Processing (NLP) is a fascinating field..." tokens = word_tokenize(text) fdist = FreqDist(tokens) print(tokens) print(fdist.most_common(10))` Sample Output: `['Natural', 'Language', 'Processing', ...] [('NLP', 2), ('and', 2), ('language', 2), ('is', 2), ('a', 1), ...]` Tokenization breaks text into words, while frequency distribution counts occurrences of each token.

Question 8

Python Code: `from tensorflow.keras.preprocessing.text import Tokenizer from tensorflow.keras.preprocessing.sequence import pad_sequences from tensorflow.keras.models import Sequential from tensorflow.keras.layers import Embedding, LSTM, Dense texts = ["I love this project", "This is an amazing experience", "I hate waiting in line", "This is the worst service", "Absolutely fantastic!"] labels = [1,1,0,0,1] tokenizer = Tokenizer() tokenizer.fit_on_texts(texts) sequences = tokenizer.texts_to_sequences(texts) X = pad_sequences(sequences, maxlen=5) model = Sequential([Embedding(input_dim=50, output_dim=8, input_length=5), LSTM(16), Dense(1, activation='sigmoid')]) model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy']) model.fit(X, labels, epochs=20)` Sample Output: `Epoch 20/20 loss: 0.45 accuracy: 1.00` The model learns to classify positive and negative sentences from the small dataset.

Question 9

Python Code: `import spacy nlp = spacy.load("en_core_web_sm") text = "Homi Jehangir Bhaba was an Indian nuclear physicist who played a key role in the development of India's atomic energy program. He was the founding director of the Tata Institute of Fundamental Research (TIFR) and was instrumental in establishing the Atomic Energy Commission of India." doc = nlp(text) for token in doc: print(token.text, token.lemma_) print("Entities:") for ent in doc.ents: print(ent.text, ent.label_)` Sample Output: `Homi -> Homi played -> play development -> development Entities: Homi Jehangir Bhaba (PERSON) India (GPE) Tata Institute of Fundamental Research (ORG) Atomic Energy Commission of India (ORG)`

Question 10

Architecture: 1. User input enters the chatbot. 2. spaCy or Stanford NLP performs tokenization, lemmatization, entity recognition, and dependency parsing. 3. Cleaned text is converted into embeddings. 4. An LSTM or GRU network processes the sequence and identifies intent, emotion, or risk level. 5. A response generation module selects an appropriate response. 6. Safety filters check for harmful or crisis-related content. Data Preprocessing: - Text cleaning - Tokenization - Stop-word removal - Lemmatization - Embedding generation - Sequence padding Why LSTM/GRU: These models capture contextual information and understand sentence meaning over time, making them suitable for conversational AI. Ethical Considerations: - Protect user privacy and data security. - Avoid biased responses. - Clearly state that the chatbot is not a replacement for professional mental-health care. - Detect self-harm or crisis-related language and escalate to human professionals when necessary. No full Python implementation is required because the question focuses mainly on system design and ethical considerations.